



FULL STACK WEB DEVELOPMENT (22CAC09N)

CASE STUDY REPORT

CineWorld — Movie Ticket Booking & Resale Platform

SUBMITTED BY

Name : A.Bheeshma Shankar & Ch.Pranav

Roll Number : 160124748093 & 160124748099

**B.E. CSM-2
V SEMESTER
III YEAR**

ACADEMIC YEAR: 2026-2027

SUBMITTED TO

Ms. G. Baby Lakshmi Prasanna



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY

(A) GANDIPET, HYDERABAD - 500075

Case Study – 1 : CineWorld - Movie Ticket Booking & Resale Platform

1. Introduction

1.1 Problem Statement

Booking movie tickets online often looks simple on the surface, but two real problems show up once a platform is actually used. First, browsing is inefficient — users need to see movies, showtimes, seat layouts, and prices together, and select seats interactively while the price updates instantly. Second, and more overlooked, once a ticket is booked there is normally no way to recover its value if plans change — the ticket simply goes unused, the buyer loses money, and other interested users can't get a seat that is technically free.

To address both problems, this case study presents CineWorld, a web-based movie ticket booking platform that also allows users to resell tickets they can no longer use to other users, turning what would be a wasted booking into a recovered sale.

1.2 Importance in Full Stack Development

CineWorld is a strong full stack development case study because it demonstrates how multiple layers of web technology work together to support both a standard booking flow and a peer-to-peer resale feature.

The project demonstrates:

- HTML5 for page structure across the catalogue, details, and booking pages
- CSS3 for styling individual UI elements
- Bootstrap 5 for responsive grid layout and pre-built components (cards, navbar, carousel, forms)
- JavaScript for logic, event handling, validation, and data types
- AJAX and the Fetch API for asynchronous movie and showtime data retrieval
- JSON for structuring movie, customer, and booking records
- ReactJS for component-based UI development
- Local Storage for persisting resale ticket listings across page refreshes

The project also demonstrates key React concepts: JSX, functional components, props, state, the `useState` hook, the `useEffect` hook, and controlled forms with validation.

1.3 Real-World Applications

The same concepts extend directly to real-world systems such as:

- Cinema chains and multiplex booking platforms
- Airline and bus seat-booking systems

- Event and concert ticketing platforms
- Resale marketplaces (e.g. secondary ticket markets)
- Any e-commerce flow involving selection, dynamic pricing, and checkout

2. Methods Compared

2.1 Traditional Ticket Booking (Box Office / Phone Booking)

Traditional movie ticket booking relies on visiting a counter in person or calling the cinema to check availability and reserve seats.

Typical operations include:

- Checking seat availability by asking the counter staff or calling
- Manually noting down the customer's seat and showtime preference
- Collecting payment in cash or by card at the counter
- Printing a physical ticket
- No way to resell a ticket if plans change — it is simply wasted
- Long queues during peak hours and popular shows

This approach works for small-scale operations but becomes inefficient as the number of shows, seats, and daily customers grows, and it offers no way to recover the value of an unused ticket.

2.2 Proposed CineWorld Web-Based Booking & Resale System

The proposed system digitizes the entire booking process and adds a resale mechanism that traditional booking has no equivalent for.

The system provides:

- Dynamic movie catalogue with posters, genres, and showtimes
- Interactive seat map with real-time price calculation
- Validated online checkout with customer details
- Instant digital ticket confirmation
- Ticket resale: listing an unused ticket for other users to buy
- Automatic listing removal once a resold ticket is purchased
- Persistent resale listings across page refresh via Local Storage

2.3 Comparison

Feature	Traditional Booking	CineWorld (Proposed)
Seat availability	Asked at counter or over phone	Live seat map on screen
Seat selection	Assigned by staff	User selects seats interactively
Price calculation	Manually quoted by staff	Calculated instantly as seats are selected
Booking confirmation	Physical printed ticket	Digital confirmation screen
Unused tickets	Lost — no recovery option	Can be resold to another user
Ticket resale	Not possible	Listed and sold through Ticket Exchange
Data handling	Paper records / phone notes	JSON data via Fetch API
Data persistence	Physical ticket stubs, registers	Local Storage in the browser
UI updates	None — static process	React re-renders instantly on state change
Accessibility	Requires visiting or calling the cinema	Available anytime from a browser

2.4 Technologies Used

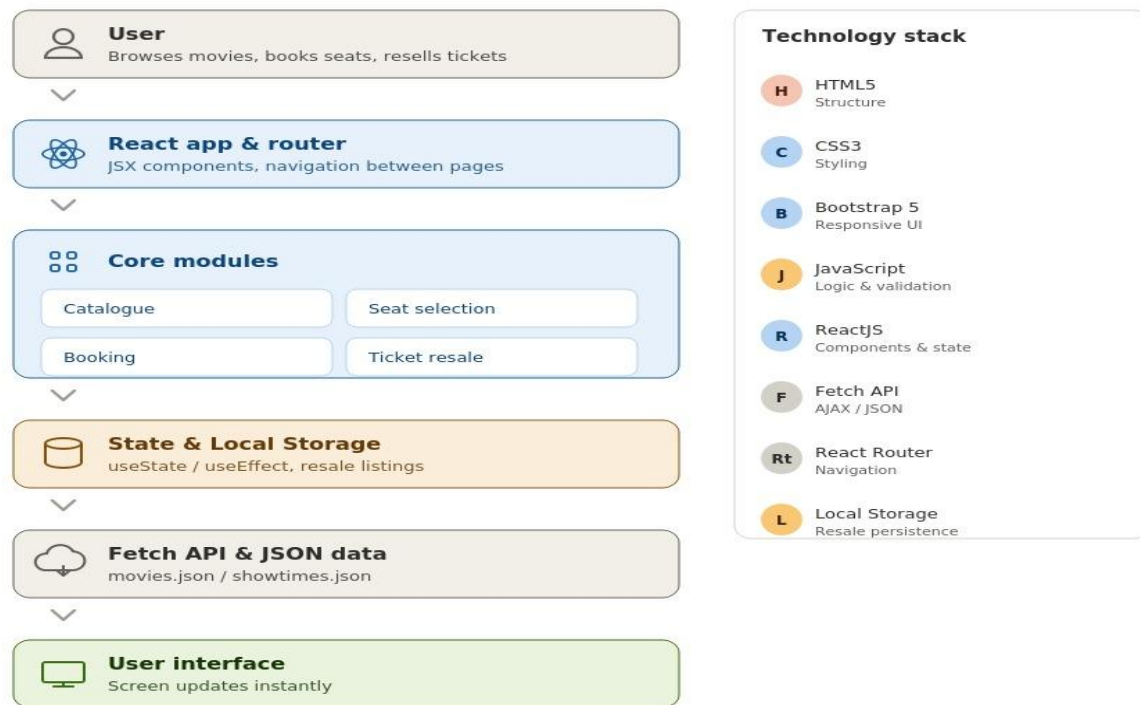
Technology	Purpose
HTML5	Structure for home, catalogue, details, and booking pages
CSS3	Styling of cards, buttons, forms, and layout spacing
Bootstrap 5	Responsive grid, carousel, navbar, cards, and form styling
JavaScript	Data types, type conversion, events, OOP, and regex validation
AJAX / Fetch API	Asynchronous retrieval of movie and showtime data
JSON	Structured representation of movie, customer, and booking records
ReactJS	Component-based UI: JSX, props, state, hooks
React Router	Navigation between Home, Movies, Details, Booking, and Confirmation
Local Storage	Persisting resale ticket listings in the browser

3. Architecture

3.1 Logical Architecture

CineWorld follows a client-side application architecture where React manages both the UI and the application state. Movie data flows in through the Fetch API as JSON; user-driven data — like seat selections and resale listings — is tracked in React state and, for resale listings, mirrored into Local Storage so it survives a page refresh.

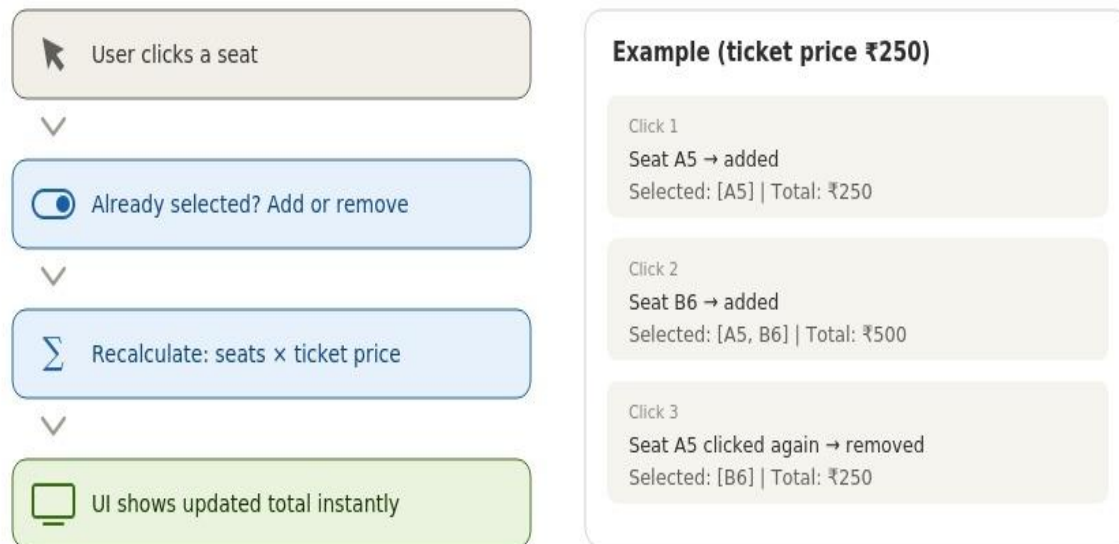
Logical Flow:



3.2 Request and Data Flow

1. The user opens a page using the navigation bar.
2. React Router displays the required page/component (Home, Movies, Booking, etc.).
3. The relevant component loads the required data.
4. The Fetch API retrieves movie/showtime JSON data asynchronously.
5. The response is converted into JavaScript objects using `response.json()`.
6. The data is stored in React state using `useState`.
7. React dynamically renders the movie catalogue, seat map, or ticket listings.
8. User actions — selecting seats, submitting the booking form, listing a ticket for resale — update application state.
9. Resale ticket listings are stored in Local Storage so they persist across refresh.

3.3 Seat Selection & Price Calculation Flow



3.4 Ticket Resale Flow

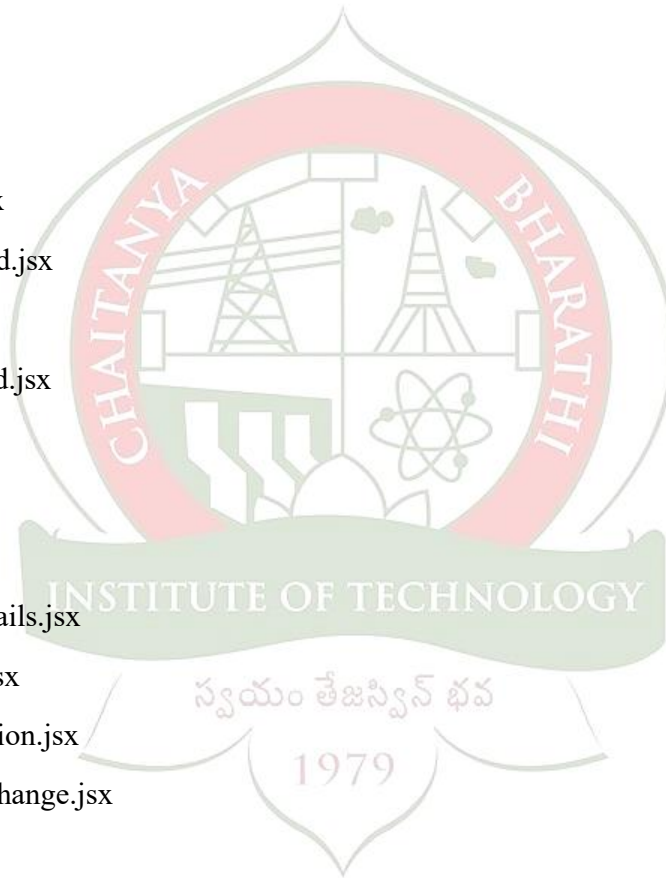


4. Implementation

4.1 Project Structure

cineworld-app/

```
|
|
|— public/
|   |— movies.json
|   |— showtimes.json
|
|— src/
|   |— components/
|       |— Navbar.jsx
|       |— MovieCard.jsx
|       |— Seat.jsx
|       |— TicketCard.jsx
|
|   |— pages/
|       |— Home.jsx
|       |— MovieDetails.jsx
|       |— Booking.jsx
|       |— Confirmation.jsx
|       |— TicketExchange.jsx
|
|   |— App.jsx
|   |— App.css
|   |— index.css
|   |— main.jsx
|
|— package.json
|— vite.config.js
```



4.2 React Component Creation

A reusable MovieCard component displays each movie in the catalogue.

```
jsx
function MovieCard({ movie }) {
  return (
    <div className="movie-card">
      <img src={movie.poster} alt={movie.title} />
      <h3>{movie.title}</h3>
      <p>Genre: {movie.genre}</p>
      <p>Showtime: {movie.showtime}</p>
    </div>
  );
}
export default MovieCard;
```

The component receives movie information through props and dynamically displays it.

4.3 Fetch API and JSON

Movie data is loaded from a JSON file using the Fetch API.

```
jsx
useEffect(() => {
  fetch("/movies.json")
    .then(response => response.json())
    .then(data => setMovies(data));
}, []);
```

Here:

fetch() - requests the JSON file

response.json() - converts the response into JavaScript data

setMovies() - stores the data in React state

useEffect() - runs the data-loading operation when the component mounts

4.4 Seat Selection Logic

```
jsx
function toggleSeat(seatId) {
  setSelectedSeats(prev =>
    prev.includes(seatId)
      ? prev.filter(id => id !== seatId)
      : [...prev, seatId]
  );
}
const totalPrice = selectedSeats.length * ticketPrice;
```

Each seat click updates the `selectedSeats` state, and the total price is recalculated instantly on every render.

4.5 Booking Form Validation

```
jsx
const isValidEmail = email =>
  /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
const isValidPhone = phone => /^[0-9]{10}$/.test(phone);
function handleSubmit(e) {
  e.preventDefault();
  if (!isValidEmail(customer.email) ||
    !isValidPhone(customer.phone)) {
    setError("Please enter a valid email and 10-digit phone
number.");
    return;
  }
  confirmBooking();
}
```

Regular expressions validate the email and phone number before the booking is confirmed.

4.6 Ticket Resale with Local Storage

```
jsx
function listTicketForResale(ticket) {
  const listings =
    JSON.parse(localStorage.getItem("resaleTickets")) || [];
  listings.push(ticket);
  localStorage.setItem("resaleTickets",
    JSON.stringify(listings));
}
function buyResaleTicket(ticketId) {
  let listings =
    JSON.parse(localStorage.getItem("resaleTickets")) || [];
  listings = listings.filter(t => t.id !== ticketId);
  localStorage.setItem("resaleTickets",
    JSON.stringify(listings));
}
```

listTicketForResale() adds a new listing to Local Storage, and **buyResaleTicket()** removes it once purchased — this is what makes listings persist across a page refresh and disappear automatically once sold.

4.7 React Router

```
jsx
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/movie/:id" element={<MovieDetails />} />
  <Route path="/booking/:id" element={<Booking />} />
  <Route path="/confirmation" element={<Confirmation />} />
  <Route path="/exchange" element={<TicketExchange />} />
</Routes>
```

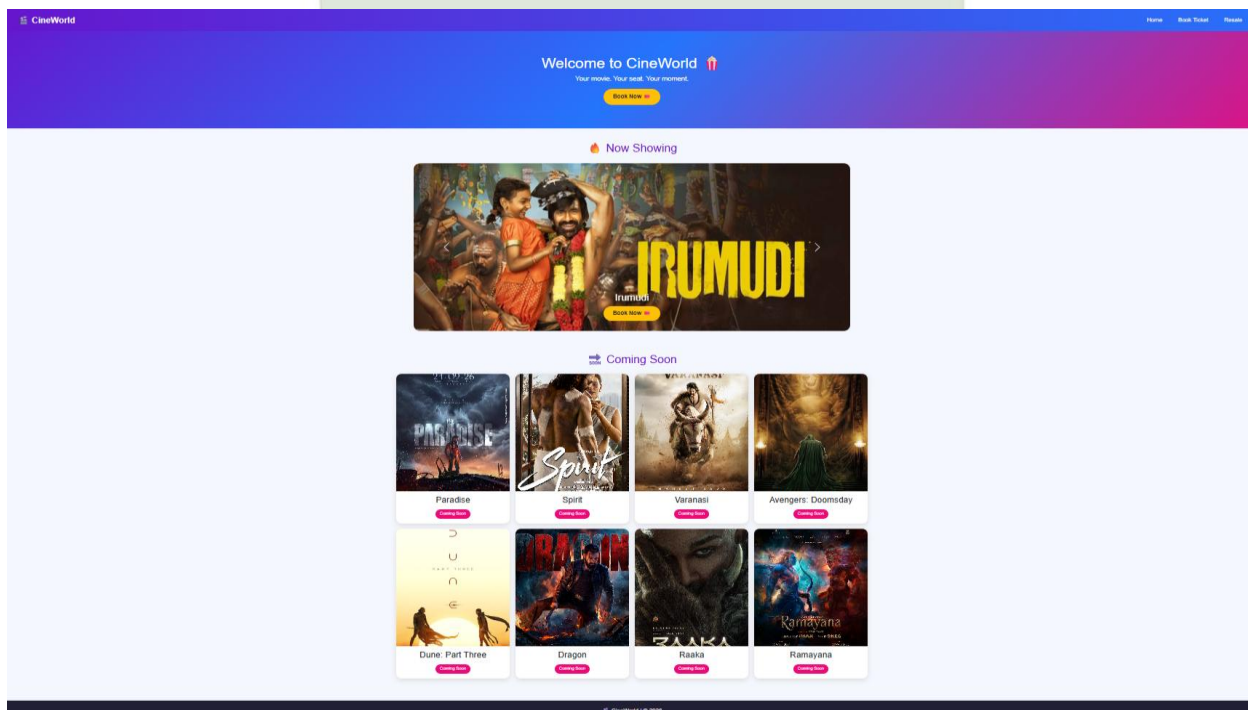
This provides navigation between the Home, Movie Details, Booking, Confirmation, and Ticket Exchange pages.

5. Results

The developed CineWorld platform successfully provides an interactive movie ticket booking and resale experience, combining a dynamic catalogue, live seat selection, validated checkout, and a working resale marketplace.

5.1 Home Page

The Home page displays the movie catalogue with posters, genres, and available showtimes, pulled in dynamically through the Fetch API from movies.json. Users can browse the full list and click into any movie to view details and proceed to booking.



5.2 Seat Selection & Price Calculation

The Booking page shows an interactive seat map. Clicking a seat toggles its selected state, and the total price recalculates instantly as seats selected $\times$ ticket price.

CineWorld

HomeBook TicketResale

Book Ticket

Movie

Irumudi - ₹200

Show Time

10:00 AM1:00 PM4:00 PM7:00 PM

Selected: 10:00 AM

Date

09/12/2026

SCREEN

Select Seats

A1	A2	A3	A4	A5	A6	A7	A8
B1	B2	B3	B4	B5	B6	B7	B8
C1	C2	C3	C4	C5	C6	C7	C8
D1	D2	D3	D4	D5	D6	D7	D8
E1	E2	E3	E4	E5	E6	E7	E8

Selected: A6, B6

Total: ₹400

Total for 2 seat(s): ₹400

Bheeshma

bheeshma@gmail.com

888888888

Confirm Booking

5.3 Ticket Exchange (Resale) Page

The Ticket Exchange page lists all tickets currently available for resale, read live from Local Storage. A seller can list an unused ticket (movie, showtime, seat, price), and it appears in this list immediately.

CineWorld

HomeBook TicketResale

Ticket Exchange

Sell your ticket if you cannot attend the show.

Bheeshma

Irumudi

10:00 AM

A6

200

List Ticket

Available Tickets

Irumudi

10:00 AM

Seat: A6

Seller: Bheeshma

Price: ₹200

Buy Ticket

5.4 Booking Confirmation

After a successful booking or resale purchase, a confirmation screen displays the movie, showtime, seat(s), and total amount paid. For a resale purchase, the corresponding listing is removed from Local Storage and disappears from the Ticket Exchange page automatically.



5.6 Result Summary

The implemented system successfully demonstrates:

- Dynamic movie catalogue display
- Interactive seat selection with live price calculation
- Validated customer booking form
- Ticket resale listing and automatic removal after purchase
- Local Storage-based persistence across page refresh
- Fetch API and JSON data handling
- React component-based development
- React state management (useState, useEffect)
- Client-side navigation with React Router

5.7 Presentation Photo



6. Conclusion

CineWorld successfully demonstrates how modern full stack web technologies can be combined into a realistic, working application. HTML5, CSS3, and Bootstrap 5 provide the structure and responsive styling; JavaScript, AJAX, and JSON handle interactivity, validation, and asynchronous data; and ReactJS ties everything into a modular, component-based architecture. The ticket resale feature — backed by Local Storage — shows how client-side persistence can support a genuinely useful secondary feature (recovering value from unused tickets) without needing a backend for a first working version.

Future Scope

The current system can be further enhanced by:

- Developing a Node.js and Express backend
- Connecting the application to MongoDB or MySQL for persistent, multi-user data
- Implementing real user authentication (JWT or session-based)
- Adding secure payment gateway integration for both booking and resale purchases
- Maintaining a complete booking and resale transaction history
- Adding real-time seat-lock synchronization to prevent double booking
- Sending email or SMS confirmations for bookings and resale purchases
- Deploying the application to a cloud platform
- Providing an admin dashboard for managing movies, showtimes, and listings

A complete production version could evolve from the current React-based prototype into a full-stack application with a dedicated backend, database, authentication, and payment processing.

INSTITUTE OF TECHNOLOGY

స్వయం తేజస్విన్ భవ

1979